Project Title

AI-Driven Network Threat Prediction and Detection System

By

Student Name: Adelaide Miguel (92301733012)

Student Name: Armando Nelvio (92301733020)

Under the guidance of

Prof. Nishith Kotak

A Project Submitted to

Marwadi University in Partial Fulfillment of the Requirements for the subject

Artificial Intelligence (01CT0616)

May 2025

MARWADI UNIVERSITY

Rajkot-Morbi Road, At & Po. Gauridad, Rajkot-360003, Gujarat, India

Table of Contents

List of Figures

List of Tables

Abstract

This project presents an Artificial Intelligence (AI)-driven system for network threat prediction and detection using advanced machine learning techniques. With the rapid growth of digital communication and increasing cybersecurity risks, traditional rule-based systems are no longer sufficient to detect evolving and unknown attack patterns. To address this challenge, the proposed system leverages datadriven approaches to automatically identify malicious activities within network traffic. The system processes network traffic data through structured stages, including data preprocessing, feature engineering, and model training. Multiple machine learning algorithms, such as Random Forest, Support Vector Machine (SVM), and Extreme Gradient Boosting (XGBoost), are implemented and evaluated using performance metrics including accuracy, precision, recall, and F1-score. To ensure reliability and robustness, Stratified K-Fold Cross-Validation is applied during model evaluation. The best-performing model is integrated into a web-based dashboard that enables real-time monitoring, data upload, visualization, and threat detection. The system is further enhanced with practical features such as secure user authentication, adaptive detection thresholds, attack classification, log export functionality, and realtime user notifications. Experimental results demonstrate that the proposed system achieves high detection accuracy and provides an efficient, scalable, and practical solution for modern cybersecurity applications. The deployment of the system confirms its usability in real-world environments, making it a valuable contribution to intelligent network security systems.

1. Introduction

In recent years, the rapid expansion of digital technologies and network-based systems has significantly increased the volume and complexity of data transmitted across networks. While this growth has enabled greater connectivity and efficiency, it has also exposed systems to a wide range of cybersecurity threats, including malware attacks, denial-of-service (DoS) attacks, and unauthorized access. As a result, ensuring network security has become a critical concern for organizations and individuals.

Traditional intrusion detection systems (IDS), which rely on predefined rules and signature-based methods, often struggle to detect new and evolving attack patterns. These systems are limited in their ability to adapt to dynamic environments, making them less effective against modern cyber threats. Consequently, there is a growing need for intelligent and adaptive solutions capable of identifying both known and unknown attacks with high accuracy.

Artificial Intelligence (AI), particularly machine learning, has emerged as a powerful approach to address these challenges. By learning patterns from historical network data, machine learning models can automatically identify anomalies and classify network activities as normal or malicious. Unlike traditional systems, AI-based approaches offer adaptability, scalability, and improved detection performance.

This project focuses on the design and development of an AI-driven network threat prediction and detection system. The proposed system utilizes advanced data preprocessing techniques, feature engineering, and multiple machine learning algorithms to analyze network traffic effectively. Furthermore, it integrates a web-based dashboard that enables real-time monitoring, visualization, and interaction, enhancing usability and practical deployment.

The main contribution of this work lies in combining machine learning-based detection with system-level implementation, resulting in a scalable, efficient, and user-friendly solution for modern cybersecurity challenges.

2. Problem Statement

The increasing volume, velocity, and complexity of network traffic have made it difficult to effectively detect malicious activities using traditional security mechanisms. Conventional intrusion detection systems, which rely on static rules and signature-based approaches, are often limited in identifying new, unknown, or evolving cyber threats. This limitation results in reduced detection accuracy, higher false alarm rates, and an inability to respond effectively to modern attack patterns.

Therefore, there is a need for an intelligent and adaptive system capable of analyzing large-scale network data and accurately distinguishing between normal and malicious behavior. The proposed solution aims to develop a machine learning-based intrusion detection system that enhances detection performance, reduces false positives and false negatives, and provides reliable, real-time threat identification suitable for practical cybersecurity environments.

3. Relevance to Artificial Intelligence

Artificial Intelligence (AI) plays a vital role in modern cybersecurity by enabling automated and intelligent analysis of large-scale network data. With the increasing complexity of cyber threats, manual and rulebased approaches are no longer sufficient to ensure effective protection. AI techniques, particularly

machine learning, allow systems to learn patterns from historical data and identify anomalies that may indicate malicious activities, including previously unseen attacks.

In this project, AI is used to build a predictive and adaptive intrusion detection system capable of improving its performance over time. Machine learning models analyze network traffic features, classify activities, and support real-time decision-making. The integration of AI not only enhances detection accuracy but also provides scalability, flexibility, and efficiency, making it highly suitable for addressing the challenges of evolving cybersecurity threats in real-world environments.

4. Objectives

The primary objective of this project is to design and develop an intelligent network threat detection system using Artificial Intelligence and machine learning techniques. The system aims to enhance cybersecurity by accurately identifying malicious network activities and providing reliable real-time insights.

The specific objectives of the project are as follows:

- To develop a machine learning-based model capable of accurately detecting malicious network traffic.
- To improve model performance through effective data preprocessing and feature engineering techniques.
- To evaluate and compare multiple machine learning algorithms in order to identify the most efficient and reliable approach for threat detection.
- To design and implement a real-time detection system integrated with a user-friendly web-based dashboard.
- To enhance system usability by incorporating visualization tools, alert mechanisms, and interactive features for better user experience.

5. Scope of the Project

This project focuses on the development of an AI-driven network threat detection system using structured network traffic datasets. The system is primarily designed to operate on the CIC-IDS dataset, which provides labeled instances of both normal (benign) and malicious network activities. The current implementation emphasizes binary classification, enabling the system to distinguish effectively between benign and attack traffic.

The proposed system is scalable and can be extended to support multi-class classification for identifying specific types of cyberattacks. It is applicable in real-world environments such as enterprise networks and cloud-based infrastructures, where continuous monitoring and threat detection are essential. However, the system relies on historical labeled data and machine learning techniques, which may limit its performance in fully unsupervised or highly dynamic environments. Despite these limitations, the system provides a strong foundation for building advanced and intelligent cybersecurity solutions.

6. Methodology

The proposed methodology follows a structured and systematic pipeline for network threat detection, with the potential to extend toward predictive analysis based on learned traffic patterns. Each stage of the pipeline is designed to ensure data quality, model efficiency, and reliable performance in detecting malicious activities.

6.1 Data Collection

The system utilizes the CIC-IDS dataset, which consists of multiple CSV files containing real-world network traffic data. The dataset includes labelled instances of both benign and malicious activities, forming a strong foundation for supervised machine learning. This data provides diverse traffic patterns necessary for training robust detection models.

6.2 Data Pre-processing

Data pre-processing is performed to ensure data quality and consistency. Missing and invalid values, such as NaN and infinite values, are removed to maintain data integrity. Traffic labels are converted into binary classes (BENIGN and ATTACK) to simplify the classification process. Additionally, relevant numerical features are selected to reduce noise and improve computational efficiency.

6.3 Feature Engineering

Feature engineering techniques are applied to enhance model performance. Low-variance features are removed to eliminate redundant or non-informative data. Logarithmic and quantile transformations are used to normalize feature distributions and reduce skewness. These steps improve model stability and enable more effective learning from the dataset.

6.4 Model Training (Detection Phase)

Multiple machine learning models are trained to detect malicious network traffic. Random Forest is used as a baseline ensemble classifier due to its robustness and interpretability. Support Vector Machine (SVM)

is applied for its effectiveness in handling high-dimensional data. Extreme Gradient Boosting (XGBoost) is utilized for its superior accuracy and performance in classification tasks. These models learn patterns from historical data to classify incoming traffic as either benign or malicious.

6.5 Model Validation

To evaluate model performance, Stratified K-Fold Cross-Validation is employed. This approach ensures that each fold maintains the same class distribution, reducing bias caused by class imbalance. It provides a reliable estimate of the model's generalization capability and ensures consistent evaluation across different data splits.

6.6 Predictive Capability (Extension)

Although the current system focuses primarily on detection, the trained models capture underlying traffic patterns that can support predictive analysis. The system can be extended to incorporate time-based models, such as Long Short-Term Memory (LSTM) networks, to forecast potential future threats and enhance proactive cybersecurity measures.

Table 1.1: Dataset Summary

Attribute Description	Attribute Description
Dataset Name	CIC-IDS Dataset
Data Format	CSV (Multiple Files)
Total Records	(Add actual number)
Number of Features	(Add number, e.g., 78+)
Data Type	Network Traffic Flow Data
Classes	BENIGN, ATTACK
Type of Learning	Supervised Learning
Feature Types	Numerical
Missing Values Handling	Removed (NaN and Infinite Values)

Preprocessing Applied	Cleaning, Label Encoding, Feature Selection
Feature Engineering	Log Transformation, Quantile Transformation
Target Variable	Traffic Label (Binary Classification)

7. AI Techniques Used

This project utilizes several Artificial Intelligence and Machine Learning techniques to ensure accurate, efficient, and reliable detection of network threats. These techniques are carefully selected to enhance model performance and support robust decision-making.

7.1 Supervised Machine Learning

The system is based on supervised learning, where models are trained using labeled network traffic data. Each data instance is associated with a known output (BENIGN or ATTACK), allowing the models to learn patterns and relationships between input features and target labels. This approach enables the system to make accurate predictions on unseen data.

7.2 Classification Algorithms

The core of the system relies on classification algorithms to distinguish between normal and malicious network activities. Ensemble-based and margin-based classifiers are employed to improve detection performance. Algorithms such as Random Forest, Support Vector Machine (SVM), and Extreme Gradient Boosting (XGBoost) analyze input features and assign them to predefined classes, ensuring high accuracy and robustness.

7.3 Feature Engineering

Feature engineering plays a crucial role in improving model effectiveness. Techniques such as feature selection, transformation, and normalization are applied to enhance data quality. By removing irrelevant and redundant features, the system reduces noise and ensures that only meaningful attributes contribute to model training.

7.4 Cross-Validation

To ensure reliable model evaluation, Stratified K-Fold Cross-Validation is used. This technique divides the dataset into multiple subsets while maintaining class distribution across each fold. It provides a consistent and unbiased estimate of model performance and helps prevent overfitting.

7.5 Ensemble Learning

Ensemble learning techniques, particularly Random Forest and XGBoost, are utilized to improve predictive performance. These methods combine multiple decision trees to produce more accurate and stable results compared to individual models. Ensemble approaches enhance generalization and reduce variance in predictions.

8. System Design and Architecture

The proposed system is designed using a modular and scalable architecture that enables efficient processing of network traffic data and accurate detection of cyber threats. The architecture integrates data processing, machine learning, and user interaction components to provide a complete end-to-end solution.

8.1 System Overview

The system follows a structured workflow in which network traffic data is collected, processed, analyzed, and visualized. Each stage of the system is interconnected, ensuring a seamless flow of data from input to output. The architecture is designed to support real-time detection while maintaining flexibility for future enhancements.

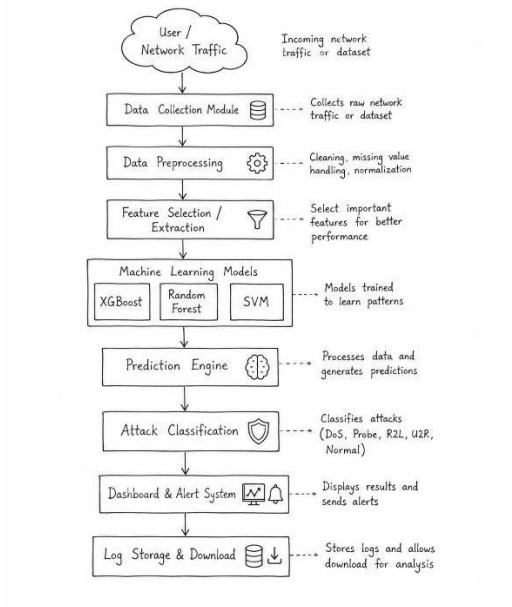


Fig. 1.1 System Architecture

8.2 Data Pipeline

The system operates through a well-defined data pipeline consisting of the following stages:

- Data Collection: Network traffic data is obtained from the CIC-IDS dataset.
- Preprocessing: Raw data is cleaned by removing missing and invalid values.
- Feature Engineering: Relevant features are selected and transformed to improve model performance.
- Model Training: Machine learning models are trained on processed data to learn traffic patterns.
- Detection: The trained model classifies incoming data as benign or malicious in real time.
- Visualization: Results are presented through an interactive dashboard for user interpretation. This

  pipeline ensures efficient handling of data and accurate threat detection throughout the system.

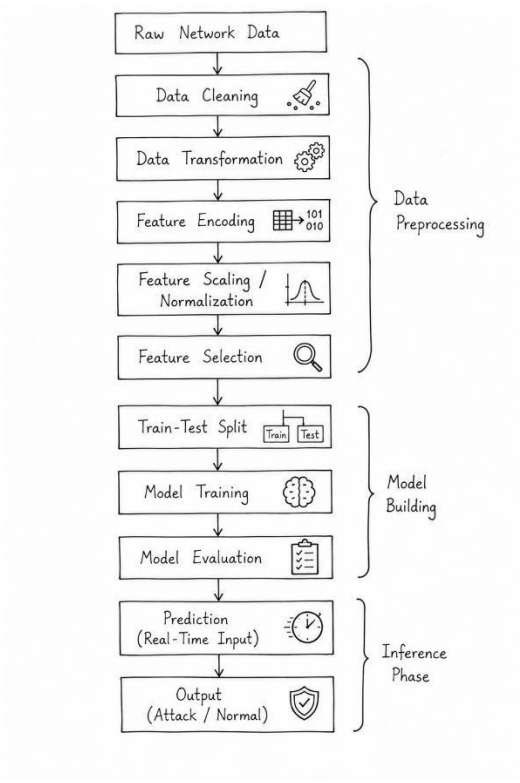


Fig. 1.2 Data Pipeline

8.3 System Modules

The system is divided into several functional modules, each responsible for a specific task:

- Data Processing Module:
  Handles data cleaning, transformation, and feature selection to prepare high-quality input for model training.
- Model Training Module:
  Responsible for training and evaluating machine learning models using the processed dataset.
- Detection Module:
  Applies the trained model to classify new or unseen network traffic and identify potential threats in real time.
- Dashboard Module:
  Provides a user-friendly interface for uploading data, visualizing results, monitoring system outputs, and receiving alerts.

8.4 System Architecture Characteristics

The architecture is designed with the following key characteristics:

- Scalability: The system can handle large volumes of network traffic and can be extended to support additional models and features.
- Modularity: Each component operates independently, allowing easy updates and maintenance.
- Real-Time Capability: The system supports real-time data analysis and threat detection.
- User Interaction: The integrated dashboard enhances usability by providing visualization, alerts, and data management features.

Overall, the system architecture ensures efficient integration of machine learning techniques with practical deployment, making it suitable for real-world cybersecurity applications.

9. Implementation

The implementation phase focuses on the development, training, and deployment of machine learning models for effective network threat detection. It also includes system integration and generation of meaningful outputs to support analysis and decision-making.

9.1 Model Training Process

The dataset is divided into training and testing sets to evaluate model performance on unseen data. Preprocessed and feature-engineered data is used to train multiple machine learning models. Each model learns patterns from historical network traffic to classify it as benign or malicious. Hyperparameter tuning is performed to optimize performance and reduce overfitting. The trained models are then evaluated using test data to assess their generalization capability.

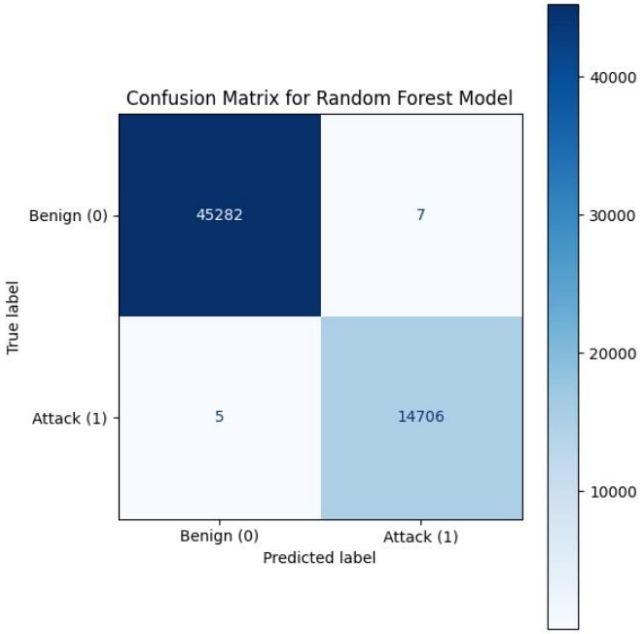
9.2 Evaluation Metrics

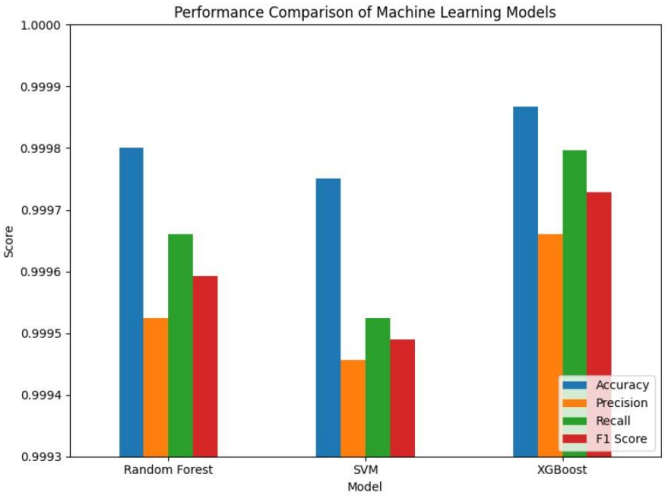
The effectiveness of the models is measured using standard performance metrics:

- Accuracy: Measures the overall correctness of predictions.
- Precision: Indicates the proportion of correctly identified attacks, reducing false positives.
- Recall: Measures the ability to detect actual attacks, reducing false negatives.
- F1-Score: Provides a balanced measure by combining precision and recall. **9.3 System Outputs**

The system generates several outputs to facilitate analysis and interpretation:

- Confusion Matrix: Displays classification results, including true positives, true negatives, false positives, and false negatives.
- Model Comparison Graph: Compares performance across different machine learning models.
- Feature Importance Plot: Identifies the most influential features contributing to predictions, especially in ensemble models.





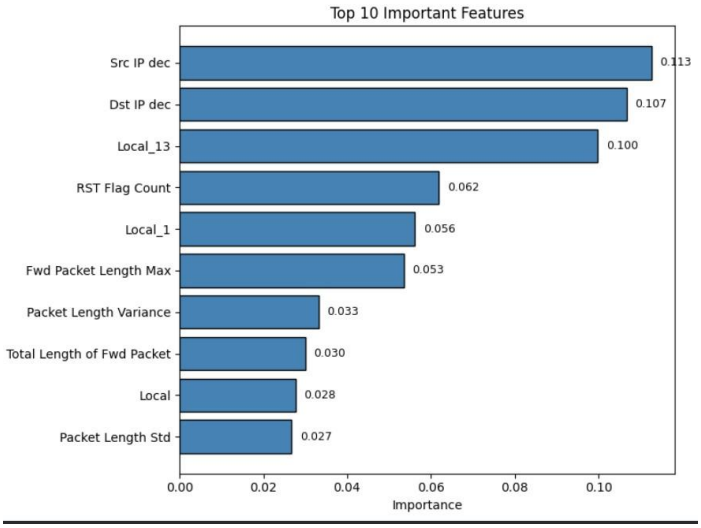


Fig. 1.3 Confusion Matrix

9.4 Additional System Features Implemented

To enhance usability and ensure practical applicability, several advanced features have been integrated into the system.

A secure user authentication mechanism has been implemented to control access to the dashboard, ensuring that only authorized users can interact with the system.

The system supports attack classification, allowing it to identify and categorize different types of malicious activities rather than limiting detection to binary classification. This enhances analytical depth and system effectiveness.

An adaptive threshold mechanism has been introduced to dynamically adjust detection sensitivity based on network conditions. This improves detection accuracy while minimizing false alarms.

A log export feature enables users to download system-generated logs for offline analysis, reporting, and auditing purposes, improving traceability and usability.

Additionally, a real-time notification system has been integrated to alert users about detected threats, system errors, and successful operations, enhancing responsiveness and situational awareness.

Finally, the system has been fully deployed and tested, confirming its functionality, stability, and suitability for real-world cybersecurity applications.

10. Evaluation

The performance of the implemented machine learning models was evaluated using standard classification metrics and cross-validation techniques to ensure accuracy, reliability, and consistency. Among the evaluated models, Extreme Gradient Boosting (XGBoost) achieved the highest accuracy of approximately 99.98%, demonstrating superior capability in detecting malicious network traffic. Random Forest and Support Vector Machine (SVM) also produced strong results, although slightly lower compared to XGBoost. The use of Stratified K-Fold Cross-Validation ensured consistent performance across different data splits and minimized bias caused by class imbalance.

The evaluation results confirm that the proposed system is highly effective in identifying network threats with high precision and recall. Overall, the system demonstrates strong generalization capability and reliability, making it suitable for real-world cybersecurity applications.

Table 1.2: Model Performance Comparison

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)
Random Forest	0.999800	0.999524	0.999660	0.999592
Support Vector Machine	0.999750	0.999456	0.999524	0.999490
XGBoost	0.999867	0.999660	0.999796	0.999728

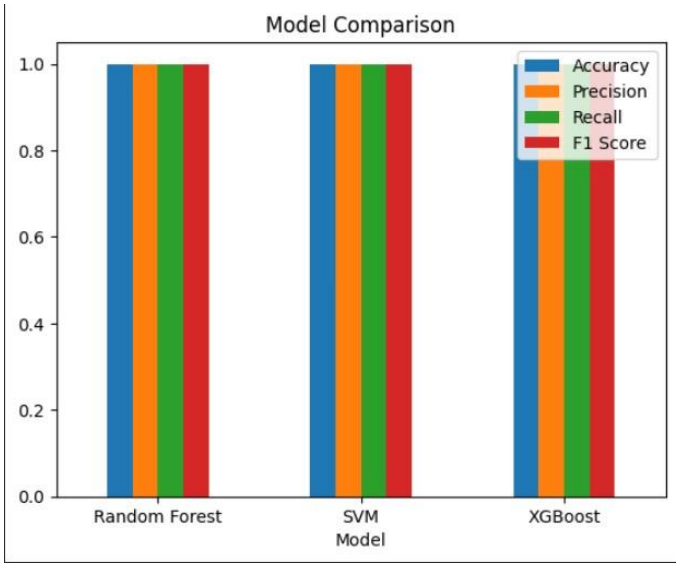


Fig. 1.4 Model Comparison Graph

11. Discussion

The proposed AI-driven network threat detection system was successfully developed and implemented, integrating data preprocessing, feature engineering, model training, evaluation, and a web-based dashboard for real-time monitoring and visualization. The system demonstrates the effectiveness of machine learning techniques in identifying malicious network activities with high accuracy and reliability.

11.1 Summary of Work

The project presents a complete end-to-end solution for network intrusion detection. From data acquisition to deployment, each stage of the system was carefully designed and implemented. The integration of multiple machine learning models and a user-friendly dashboard enhances both analytical capability and practical usability.

11.2 Challenges Faced

Several challenges were encountered during the development process. Handling large-scale network traffic datasets required efficient data processing techniques. Class imbalance between benign and attack data posed difficulties in achieving balanced model performance. Additionally, preventing overfitting while maintaining high accuracy required careful model tuning and validation.

11.3 Success Criteria

The project successfully achieved its intended objectives. High detection accuracy was obtained using advanced machine learning models, particularly XGBoost. A functional and interactive dashboard was developed, enabling real-time monitoring and user interaction. The system effectively demonstrates the capability to detect network threats in a practical environment.

11.4 Limitations

Despite its strong performance, the system has certain limitations. It currently supports only binary classification, distinguishing between benign and attack traffic. The reliance on labeled datasets limits its applicability in fully unsupervised environments. Furthermore, training and processing large datasets can be computationally intensive, requiring significant resources.

Overall, the discussion highlights that while the system is highly effective and practical, there is scope for further enhancement to improve flexibility, scalability, and adaptability.

12. Conclusion

This project successfully developed an AI-driven network threat prediction and detection system capable of identifying malicious network activities with high accuracy and efficiency. By leveraging machine learning techniques such as Random Forest, Support Vector Machine (SVM), and XGBoost, the system effectively analyzes network traffic data and distinguishes between benign and malicious behavior. The use of structured data preprocessing, feature engineering, and robust validation methods ensures reliable and consistent model performance.

The integration of a web-based dashboard enhances the practical usability of the system by enabling realtime monitoring, visualization, and user interaction. Additional features such as secure authentication, adaptive thresholding, attack classification, log export, and real-time notifications further improve system functionality and applicability. Overall, the proposed solution demonstrates strong potential for real-world deployment, providing a scalable, intelligent, and effective approach to modern cybersecurity challenges.

13. Future Work

While the proposed system demonstrates strong performance in detecting network threats, several enhancements can be explored to further improve its capabilities and applicability.

Future work may focus on extending the system from binary classification to multi-class classification, enabling the identification of specific types of cyberattacks such as DoS, phishing, and malware. This would provide deeper insights and improve the system’s analytical capabilities.

The integration of real-time packet capture mechanisms can be implemented to allow live monitoring of network traffic, making the system more suitable for deployment in dynamic environments. Additionally, incorporating deep learning techniques, such as Convolutional Neural Networks (CNN) and Long ShortTerm Memory (LSTM) networks, can enhance pattern recognition and enable temporal analysis for predictive threat detection.

Further improvements may include deploying the system on cloud platforms to ensure scalability, availability, and efficient handling of large-scale data. Enhancing security features, optimizing computational efficiency, and integrating automated response mechanisms can also contribute to building a more robust and intelligent cybersecurity solution.

Overall, these enhancements aim to transform the system into a more comprehensive, adaptive, and scalable platform for advanced network security applications.

 # Updates Made to the System

* Added real-time traffic monitoring using Python packet sniffing.

* Integrated Wireshark for packet visualization and traffic verification.

Connected the IDS and Wireshark to monitor the same network interface simultaneously

Added a traffic generator for simulating:

* benign traffic,

* port scans,

* DDoS traffic,

* and ICMP traffic.

Improved the machine learning detection workflow for live traffic analysis.

Added severity mapping for detected attacks:

* INFO,

* HIGH,

* CRITICAL.

Connected the IDS predictions to the dashboard for real-time monitoring.

Enabled dashboard visualization of:

attack types,

severity levels,

timestamps,

and suspicious activity.

Added replay and simulation support for testing the ML pipeline.

Validated the system using:

unit tests,

integration tests,

system tests,

and validation tests.

Improved the overall IDS-SIEM architecture by combining:

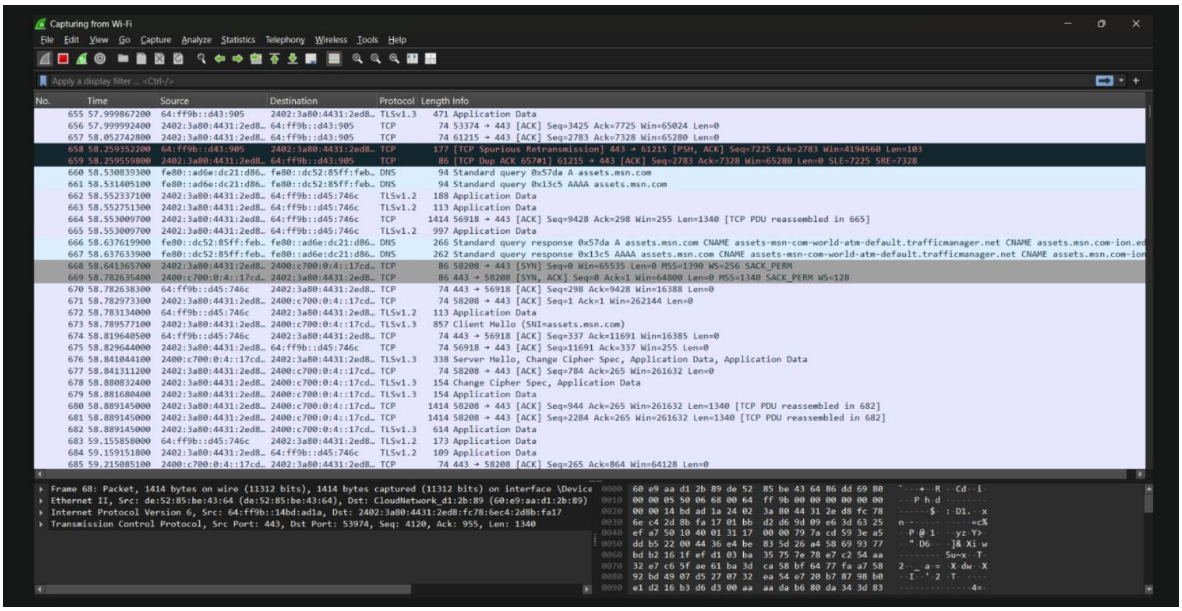
live packet capture,

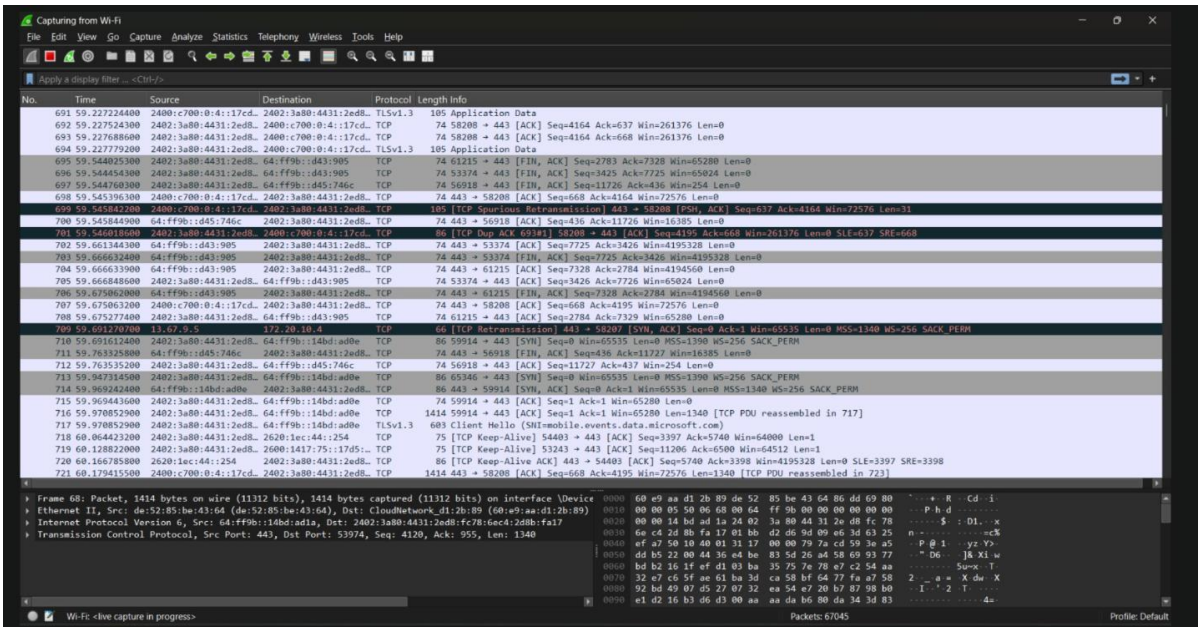
machine learning classification,

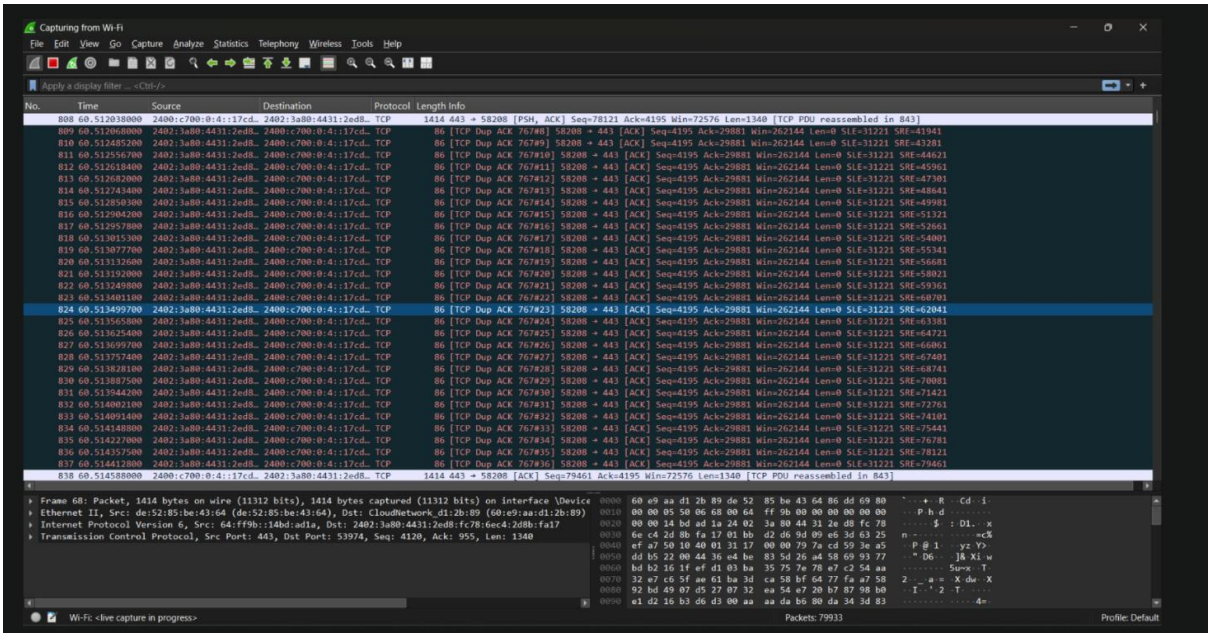
traffic simulation,

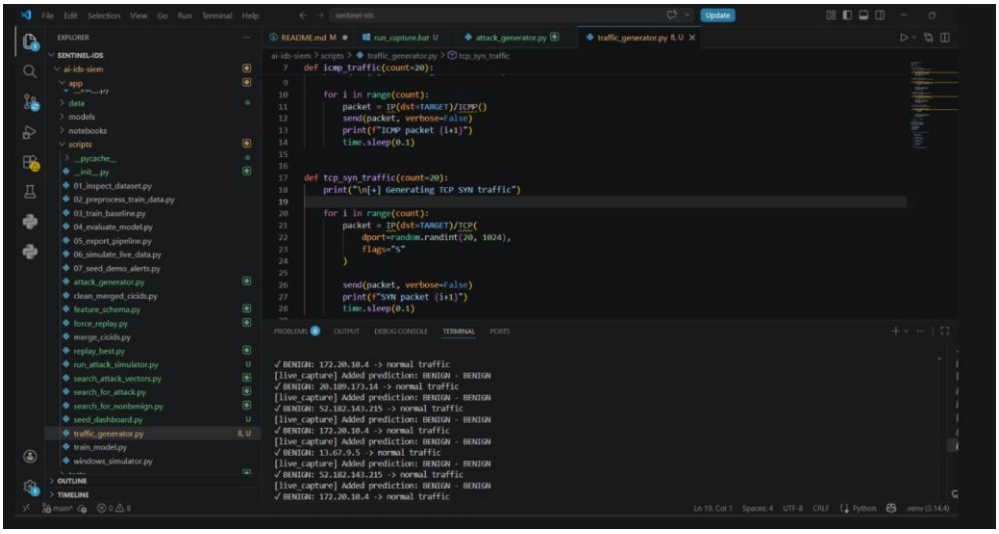
dashboard monitoring,

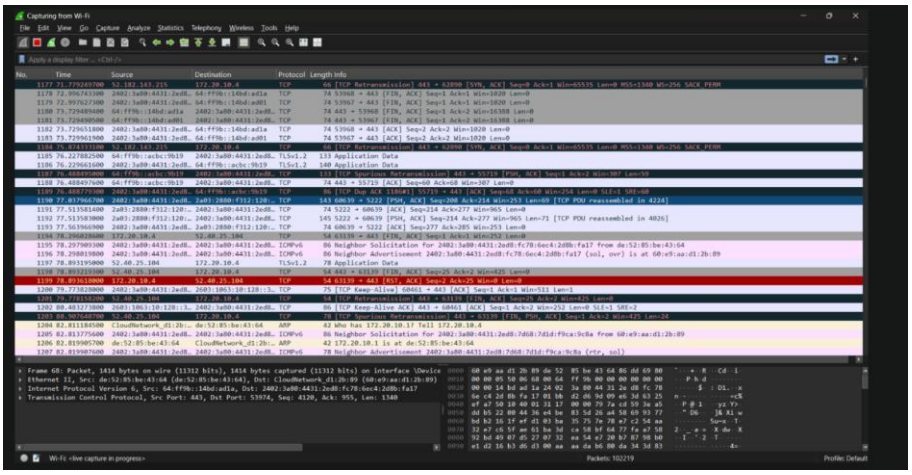
and Wireshark analysis.




Capturing from Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
691 59.227224400 2400:c700:0:4::17cd… 2402:3a80:4431:2ed8… TLSv1.3 105 Application Data
692 59.227524300 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 74 58208 → 443 [ACK] Seq=4164 Ack=637 Win=261376 Len=0
693 59.227688600 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 74 58208 → 443 [ACK] Seq=4164 Ack=668 Win=261376 Len=0
694 59.227779200 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TLSv1.3 105 Application Data
695 59.544025300 2402:3a80:4431:2ed8… 64:ff9b::d43:905 TCP 74 61215 → 443 [FIN, ACK] Seq=2783 Ack=7328 Win=65280 Len=0
696 59.544454300 2402:3a80:4431:2ed8… 64:ff9b::d43:905 TCP 74 53374 → 443 [FIN, ACK] Seq=3425 Ack=7725 Win=65024 Len=0
697 59.544760300 2402:3a80:4431:2ed8… 64:ff9b::d45:746c TCP 74 56918 → 443 [FIN, ACK] Seq=11726 Ack=436 Win=254 Len=0
698 59.545396300 2400:c700:0:4::17cd… 2402:3a80:4431:2ed8… TCP 74 443 → 58208 [ACK] Seq=668 Ack=4164 Win=72576 Len=0
699 59.545842200 2400:c700:0:4::17cd… 2402:3a80:4431:2ed8… TCP 105 [TCP Spurious Retransmission] 443 → 58208 [PSH, ACK] Seq=637 Ack=4164 Win=72576 Len=31
700 59.545844900 64:ff9b::d45:746c 2402:3a80:4431:2ed8… TCP 74 443 → 56918 [ACK] Seq=436 Ack=11726 Win=16385 Len=0
701 59.546018600 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 69301] 58208 → 443 [ACK] Seq=4195 Ack=668 Win=261376 Len=0 SLE=637 SRE=668
702 59.661344300 64:ff9b::d43:905 2402:3a80:4431:2ed8… TCP 74 443 → 53374 [ACK] Seq=7725 Ack=3426 Win=4195328 Len=0
703 59.666632400 64:ff9b::d43:905 2402:3a80:4431:2ed8… TCP 74 443 → 53374 [FIN, ACK] Seq=7725 Ack=3426 Win=4195328 Len=0
704 59.666633900 64:ff9b::d43:905 2402:3a80:4431:2ed8… TCP 74 443 → 61215 [ACK] Seq=7328 Ack=2784 Win=4194560 Len=0
705 59.666848600 2402:3a80:4431:2ed8… 64:ff9b::d43:905 TCP 74 53374 → 443 [ACK] Seq=3426 Ack=7726 Win=65024 Len=0
706 59.675062900 64:ff9b::d43:905 2402:3a80:4431:2ed8… TCP 74 443 → 61215 [FIN, ACK] Seq=7328 Ack=2784 Win=4194560 Len=0
707 59.675063200 2400:c700:0:4::17cd… 2402:3a80:4431:2ed8… TCP 74 443 → 58208 [ACK] Seq=668 Ack=4195 Win=72576 Len=0
708 59.675277400 2402:3a80:4431:2ed8… 64:ff9b::d43:905 TCP 74 61215 → 443 [ACK] Seq=2784 Ack=7329 Win=65280 Len=0
709 59.691270700 13.67.9.5 172.20.10.4 TCP 66 [TCP Retransmission] 443 → 58207 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1340 WS=256 SACK_PERM
710 59.691612400 2402:3a80:4431:2ed8… 64:ff9b::14bd:ad0e TCP 86 59914 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1390 WS=256 SACK_PERM
711 59.763925800 64:ff9b::d45:746c 2402:3a80:4431:2ed8… TCP 74 443 → 56918 [FIN, ACK] Seq=436 Ack=11727 Win=16385 Len=0
712 59.763953200 2402:3a80:4431:2ed8… 64:ff9b::d45:746c TCP 74 56918 → 443 [ACK] Seq=11727 Ack=437 Win=254 Len=0
713 59.947314500 64:ff9b::14bd:ad0e 2402:3a80:4431:2ed8… TCP 86 [TCP Retransmission] 443 → 59914 [SYN, ACK] Seq=0 Win=65535 Len=0 MSS=1390 WS=256 SACK_PERM
714 59.969242400 64:ff9b::14bd:ad0e 2402:3a80:4431:2ed8… TCP 86 443 → 59914 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1340 WS=256 SACK_PERM
715 59.969443600 2402:3a80:4431:2ed8… 64:ff9b::14bd:ad0e TCP 74 59914 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
716 59.970852900 2402:3a80:4431:2ed8… 64:ff9b::14bd:ad0e TCP 1414 59914 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=1340 [TCP PDU reassembled in 717]
717 59.970852900 2402:3a80:4431:2ed8… 64:ff9b::14bd:ad0e TLSv1.3 603 Client Hello (SNI=mobile.events.data.microsoft.com)
718 60.064423200 2402:3a80:4431:2ed8… 2620:1ec:44::254 TCP 75 [TCP Keep-Alive] 54403 → 443 [ACK] Seq=3397 Ack=5740 Win=64000 Len=1
719 60.128822000 2402:3a80:4431:2ed8… 2600:1417:75::17d5:… TCP 75 [TCP Keep-Alive] 53243 → 443 [ACK] Seq=11206 Ack=6500 Win=64512 Len=1
720 60.166785800 2620:1ec:44::254 2402:3a80:4431:2ed8… TCP 86 [TCP Keep-Alive ACK] 443 → 54403 [ACK] Seq=5740 Ack=3398 Win=4195328 Len=0 SLE=3397 SRE=3398
721 60.179415500 2400:c700:0:4::17cd… 2402:3a80:4431:2ed8… TCP 1414 443 → 58208 [ACK] Seq=668 Ack=4195 Win=72576 Len=1340 [TCP PDU reassembled in 723]
Frame 68: Packet, 1414 bytes on wire (11312 bits), 1414 bytes captured (11312 bits) on interface \Device
Ethernet II, Src: de:52:85:be:43:64 (de:52:85:be:43:64), Dst: CloudNetwork_d1:2b:89 (60:e9:aa:d1:2b:89)
Internet Protocol Version 6, Src: 64:ff9b::14bd:ad1a, Dst: 2402:3a80:4431:2ed8:fc78:6ec4:2d8b:fa17
Transmission Control Protocol, Src Port: 443, Dst Port: 53974, Seq: 4120, Ack: 955, Len: 1340
Wi-Fi: <live capture in progress>
Packets: 67045
Profile: Default

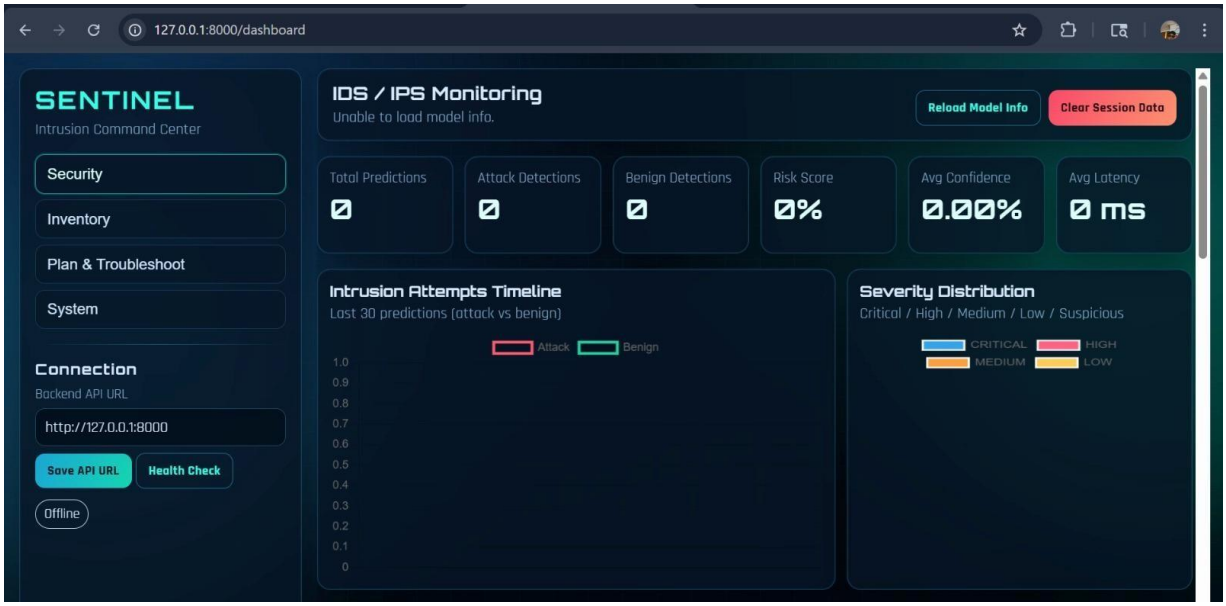

Capturing from Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
808 60.512038000 2400:c700:0:4::17cd… 2402:3a80:4431:2ed8… TCP 1414 443 → 58208 [PSH, ACK] Seq=78121 Ack=4195 Win=72576 Len=1340 [TCP PDU reassembled in 843]
809 60.512068000 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#8] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=41941
810 60.512485200 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#9] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=43281
811 60.512556700 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#10] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=44621
812 60.512618400 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#11] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=45961
813 60.512682000 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#12] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=47301
814 60.512743400 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#13] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=48641
815 60.512850300 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#14] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=49981
816 60.512904200 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#15] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=51321
817 60.512957800 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#16] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=52661
818 60.513015300 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#17] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=54001
819 60.513077700 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#18] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=55341
820 60.513132600 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#19] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=56681
821 60.513192000 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#20] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=58021
822 60.513249800 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#21] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=59361
823 60.513401100 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#22] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=60701
824 60.513499700 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#23] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=62041
825 60.513565800 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#24] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=63381
826 60.513625400 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#25] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=64721
827 60.513699700 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#26] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=66061
828 60.513757400 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#27] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=67401
829 60.513828100 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#28] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=68741
830 60.513887500 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#29] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=70081
831 60.513944200 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#30] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=71421
832 60.514002100 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#31] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=72761
833 60.514091400 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#32] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=74101
834 60.514148800 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#33] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=75441
835 60.514227000 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#34] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=76781
836 60.514357500 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#35] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=78121
837 60.514412800 2402:3a80:4431:2ed8… 2400:c700:0:4::17cd… TCP 86 [TCP Dup ACK 767#36] 58208 → 443 [ACK] Seq=4195 Ack=29881 Win=262144 Len=0 SLE=31221 SRE=79461
838 60.514588000 2400:c700:0:4::17cd… 2402:3a80:4431:2ed8… TCP 1414 443 → 58208 [ACK] Seq=79461 Ack=4195 Win=72576 Len=1340 [TCP PDU reassembled in 843]
Frame 68: Packet, 1414 bytes on wire (11312 bits), 1414 bytes captured (11312 bits) on interface \Device
Ethernet II, Src: de:52:85:be:43:64 (de:52:85:be:43:64), Dst: CloudNetwork_d1:2b:89 (60:e9:aa:d1:2b:89)
Internet Protocol Version 6, Src: 64:ff9b::14bd:ad1a, Dst: 2402:3a80:4431:2ed8:fc78:6ec4:2d8b:fa17
Transmission Control Protocol, Src Port: 443, Dst Port: 53974, Seq: 4120, Ack: 955, Len: 1340
Wi-Fi: <live capture in progress>
Packets: 79933
Profile: Default

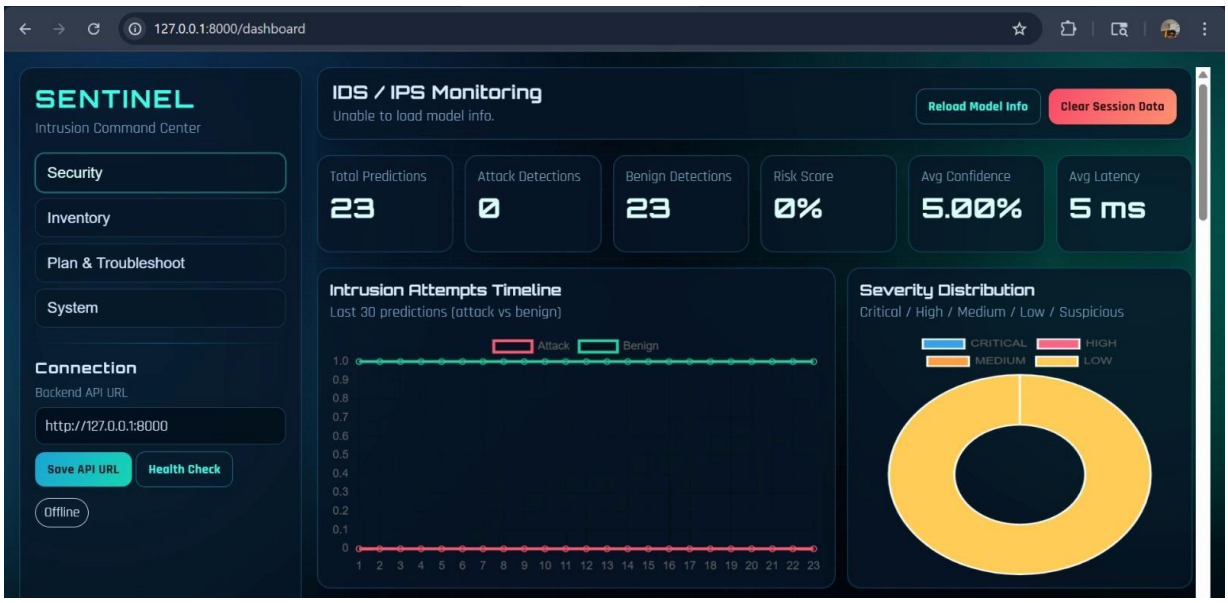

def icmp_traffic(count=20):
for i in range(count):
packet = IP(dst=TARGET)/ICMP()
send(packet, verbose=False)
print(f"ICMP packet {i+1}")
time.sleep(0.1)
def tcp_syn_traffic(count=20):
print("\n[+] Generating TCP SYN traffic")
for i in range(count):
packet = IP(dst=TARGET)/TCP(
dport=random.randint(20, 1024),
flags="S"
)
send(packet, verbose=False)
print(f"SYN packet {i+1}")
time.sleep(0.1)
✓ BENIGN: 172.20.10.4 -> normal traffic
[live_capture] Added prediction: BENIGN - BENIGN
✓ BENIGN: 20.189.173.14 -> normal traffic
[live_capture] Added prediction: BENIGN - BENIGN
✓ BENIGN: 52.182.143.215 -> normal traffic
[live_capture] Added prediction: BENIGN - BENIGN
✓ BENIGN: 172.20.10.4 -> normal traffic
[live_capture] Added prediction: BENIGN - BENIGN
✓ BENIGN: 13.67.9.5 -> normal traffic
[live_capture] Added prediction: BENIGN - BENIGN
✓ BENIGN: 52.182.143.215 -> normal traffic
[live_capture] Added prediction: BENIGN - BENIGN
✓ BENIGN: 172.20.10.4 -> normal traffic

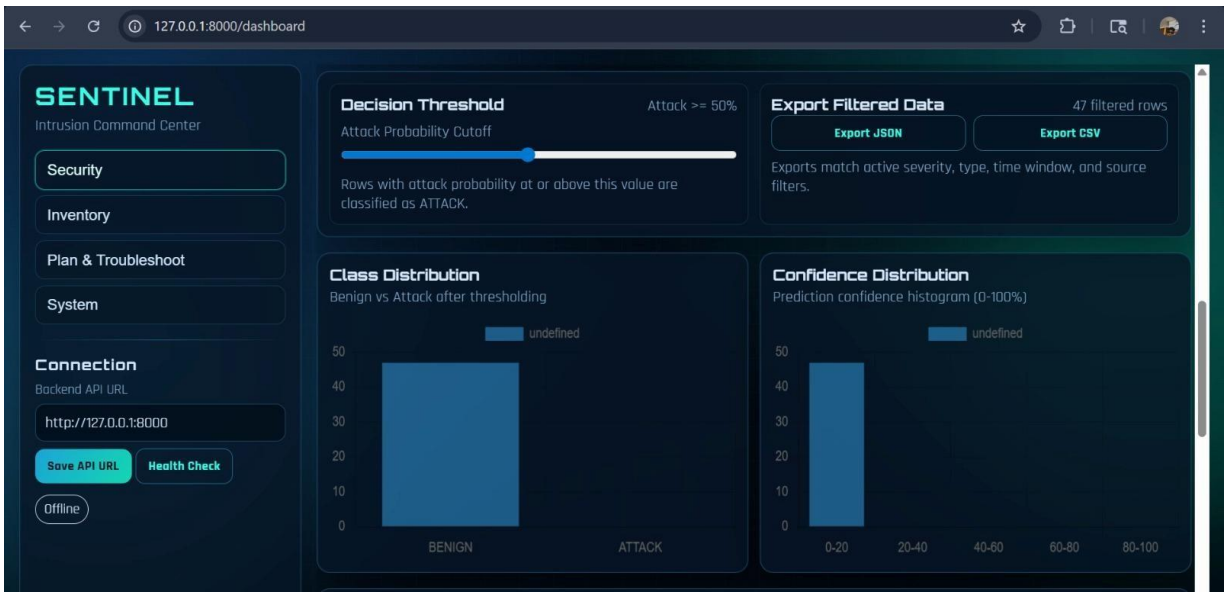

Capturing from Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1177 71.779249700 52.182.143.215 172.20.10.4 TCP 66 [TCP Retransmission] 443 → 62890 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1340 WS=256 SACK_PERM
1178 72.996743300 2402:3a80:4431:2ed8… 64:ff9b::140d:ad1a TCP 74 53968 → 443 [FIN, ACK] Seq=1 Ack=1 Win=1020 Len=0
1179 72.997627300 2402:3a80:4431:2ed8… 64:ff9b::140d:ad01 TCP 74 53967 → 443 [FIN, ACK] Seq=1 Ack=1 Win=1020 Len=0
1180 73.729489600 64:ff9b::140d:ad1a 2402:3a80:4431:2ed8… TCP 74 443 → 53968 [FIN, ACK] Seq=1 Ack=2 Win=16388 Len=0
1181 73.729490500 64:ff9b::140d:ad01 2402:3a80:4431:2ed8… TCP 74 443 → 53967 [FIN, ACK] Seq=1 Ack=2 Win=16388 Len=0
1182 73.729651800 2402:3a80:4431:2ed8… 64:ff9b::140d:ad1a TCP 74 53968 → 443 [ACK] Seq=2 Ack=2 Win=1020 Len=0
1183 73.729961900 2402:3a80:4431:2ed8… 64:ff9b::140d:ad01 TCP 74 53967 → 443 [ACK] Seq=2 Ack=2 Win=1020 Len=0
1184 75.874331100 52.182.143.215 172.20.10.4 TCP 66 [TCP Retransmission] 443 → 62890 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1340 WS=256 SACK_PERM
1185 76.227882500 64:ff9b::achc:9b19 2402:3a80:4431:2ed8… TLSv1.2 133 Application Data
1186 76.229651600 2402:3a80:4431:2ed8… 64:ff9b::achc:9b19 TLSv1.2 140 Application Data
1187 76.468495000 64:ff9b::achc:9b19 2402:3a80:4431:2ed8… TCP 133 [TCP Spurious Retransmission] 443 → 55719 [PSH, ACK] Seq=1 Ack=2 Win=307 Len=59
1188 76.468497600 64:ff9b::achc:9b19 2402:3a80:4431:2ed8… TCP 74 443 → 55719 [ACK] Seq=60 Ack=68 Win=307 Len=0
1189 76.488779300 2402:3a80:4431:2ed8… 64:ff9b::achc:9b19 TCP 86 [TCP Dup ACK 1186#1] 55719 → 443 [ACK] Seq=68 Ack=60 Win=254 Len=0 SLE=1 SRE=60
1190 77.017966700 2402:3a80:4431:2ed8… 2a03:2880:f312:120… TCP 143 60639 → 5222 [PSH, ACK] Seq=208 Ack=214 Win=253 Len=69 [TCP PDU reassembled in 4224]
1191 77.513581400 2a03:2880:f312:120… 2402:3a80:4431:2ed8… TCP 74 5222 → 60639 [ACK] Seq=214 Ack=277 Win=965 Len=0
1192 77.513583000 2a03:2880:f312:120… 2402:3a80:4431:2ed8… TCP 145 5222 → 60639 [PSH, ACK] Seq=214 Ack=277 Win=965 Len=71 [TCP PDU reassembled in 4026]
1193 77.563966900 2402:3a80:4431:2ed8… 2a03:2880:f312:120… TCP 74 60639 → 5222 [ACK] Seq=277 Ack=285 Win=253 Len=0
1194 78.290818500 172.20.10.4 52.40.25.104 TCP 54 63139 → 443 [FIN, ACK] Seq=1 Ack=1 Win=258 Len=0
1195 78.297909300 2402:3a80:4431:2ed8… 2402:3a80:4431:2ed8… ICMPv6 86 Neighbor Solicitation for 2402:3a80:4431:2ed8:fc78:6ec4:2d8b:fa17 from de:52:85:be:43:64
1196 78.298019800 2402:3a80:4431:2ed8… 2402:3a80:4431:2ed8… ICMPv6 86 Neighbor Advertisement 2402:3a80:4431:2ed8:fc78:6ec4:2d8b:fa17 (sol, ovr) is at 60:e9:aa:d1:2b:89
1197 78.893195000 52.40.25.104 172.20.10.4 TLSv1.2 78 Application Data
1198 78.893215300 52.40.25.104 172.20.10.4 TCP 54 443 → 63139 [FIN, ACK] Seq=25 Ack=2 Win=435 Len=0
1199 78.953638800 172.20.10.4 52.40.25.104 TCP 54 63139 → 443 [RST, ACK] Seq=2 Ack=25 Win=0 Len=0
1200 79.773828000 2402:3a80:4431:2ed8… 2603:1063:10:128::3… TCP 75 [TCP Keep-Alive] 60461 → 443 [ACK] Seq=1 Ack=1 Win=511 Len=1
1201 79.773915200 52.40.25.104 172.20.10.4 TCP 54 [TCP Retransmission] 443 → 63139 [FIN, ACK] Seq=25 Ack=2 Win=435 Len=0
1202 80.481273800 2603:1063:10:128::3… 2402:3a80:4431:2ed8… TCP 86 [TCP Keep-Alive ACK] 443 → 60461 [ACK] Seq=1 Ack=2 Win=252 Len=0 SLE=1 SRE=2
1203 80.707648700 52.40.25.104 172.20.10.4 TCP 78 [TCP Spurious Retransmission] 443 → 63139 [FIN, PSH, ACK] Seq=1 Ack=2 Win=425 Len=24
1204 82.811186500 CloudNetwork_d1:2b:… de:52:85:be:43:64 ARP 42 Who has 172.20.10.1? Tell 172.20.10.4
1205 82.813775600 2402:3a80:4431:2ed8… 2402:3a80:4431:2ed8… ICMPv6 86 Neighbor Solicitation for 2402:3a80:4431:2ed8:7d1d:f9ca:9c8a from 60:e9:aa:d1:2b:89
1206 82.819905700 de:52:85:be:43:64 CloudNetwork_d1:2b:… ARP 42 172.20.10.1 is at de:52:85:be:43:64
1207 82.819907600 2402:3a80:4431:2ed8… 2402:3a80:4431:2ed8… ICMPv6 78 Neighbor Solicitation for 2402:3a80:4431:2ed8:7d6d:7d1d:f9ca:9c8a (ptr, snl)
Frame 68: Packet, 1414 bytes on wire (11312 bits), 1414 bytes captured (11312 bits) on interface \Device
Ethernet II, Src: de:52:85:be:43:64 (de:52:85:be:43:64), Dst: CloudNetwork_d1:2b:89 (60:e9:aa:d1:2b:89)
Internet Protocol Version 6, Src: 64:ff9b::140d:ad1a, Dst: 2402:3a80:4431:2ed8:fc78:6ec4:2d8b:fa17
Transmission Control Protocol, Src Port: 443, Dst Port: 53974, Seq: 4120, Ack: 955, Len: 1340
Wi-Fi: <live capture in progress>
Packets: 102219
Profile: Default

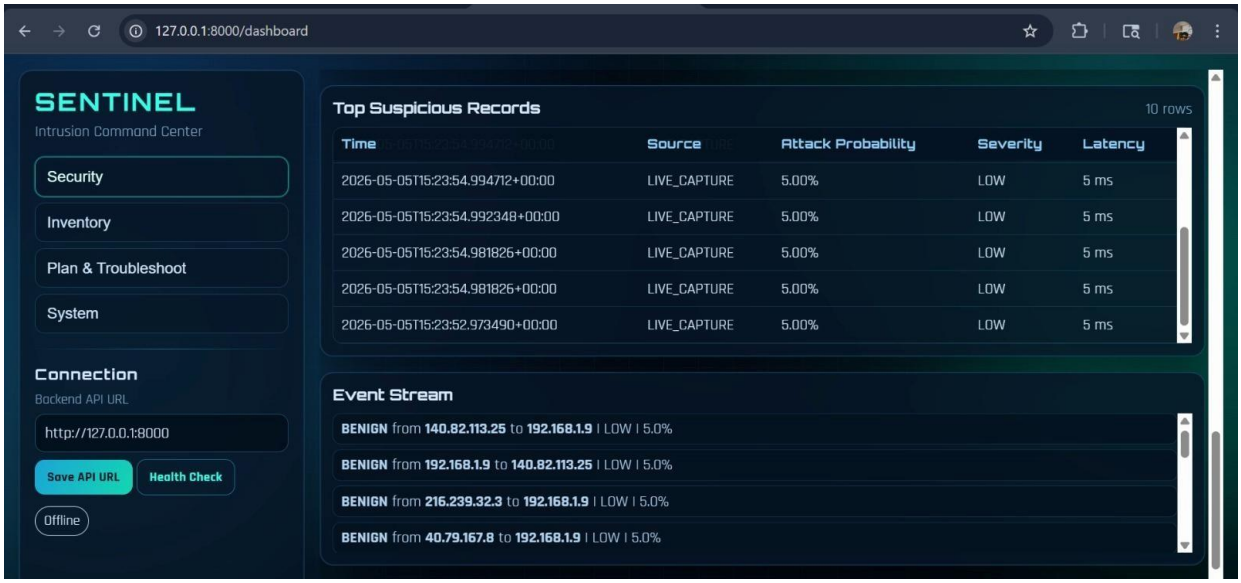
14. Appendix

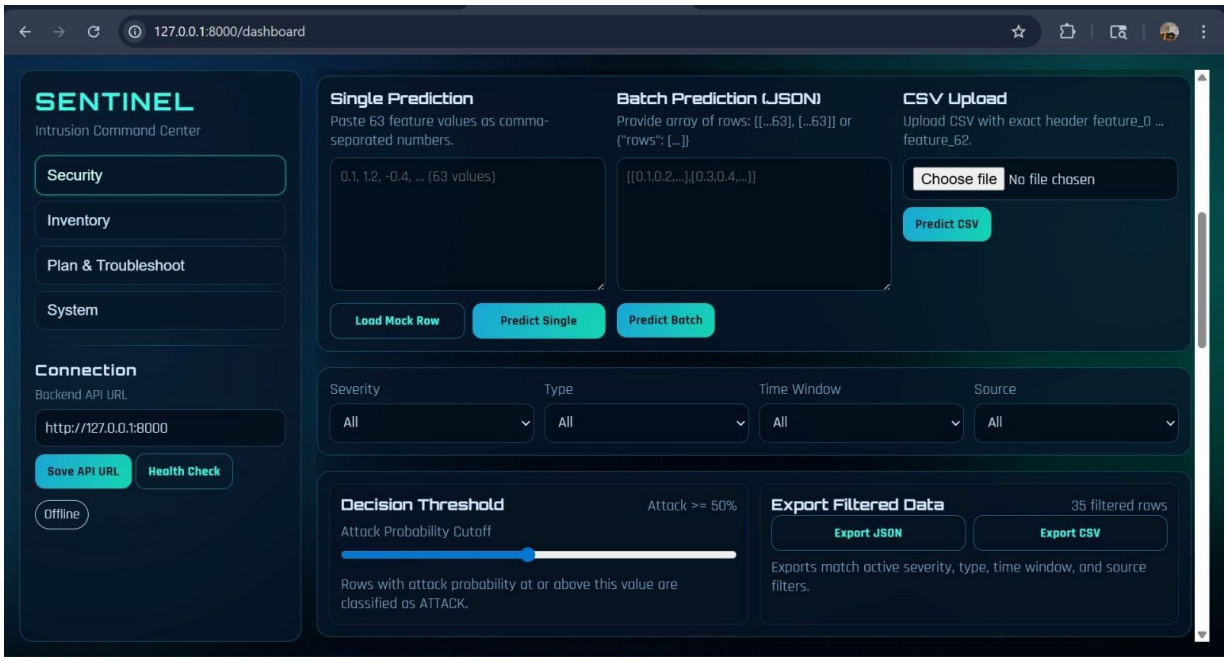
System:











Links:

Github: https://github.com/Amoiane23/sentinel-ids.git

Youtube: https://youtu.be/tE6k9RRrgaA

Cross-validation:

https://colab.research.google.com/drive/15Ofkk6ATHL3Nz96Pa83m7N1PrmUo1rep?usp=sharing

15. Full Code

```
16.      from google.colab import drive
```

```
17.      drive.mount('/content/drive')
18.      import pandas as pd
19.      import numpy as np
20.      import glob
21.      import matplotlib.pyplot as plt
22.      import seaborn as sns
23.
```

```
25.      import glob
26.
27.      path = "/content/drive/MyDrive/archiveData/*.csv"
28.      files = glob.glob(path)
29.
30.      df = pd.concat([pd.read_csv(f) for f in files], ignore_index=True)
31.
32.      print("Dataset Shape:", df.shape)
33.      import numpy as np
34.
35.      df.replace([np.inf, -np.inf], np.nan, inplace=True)
36.      df.dropna(inplace=True)
37.
38.   print("After Cleaning:", df.shape) 39.  df
= df.sample(n=300000, random_state=42)
40.
41.      print("Sampled Shape:", df.shape)
42.      df.info()
43.      df.describe()
44.      df['Label'] = df['Label'].apply(lambda x: 'Attack' if x != 'BENIGN'
         else 'BENIGN')
45.      df['Label'] = df['Label'].map({'BENIGN': 0, 'Attack': 1})
46.      import matplotlib.pyplot as plt
47.
48.      plt.figure(figsize=(6,4))
49.      df['Label'].value_counts().plot(kind='bar')
50.      plt.title("Normal vs Attack Distribution")
51.      plt.show()
52.      X = df.drop('Label', axis=1)
53.      y = df['Label']
54.      X = X.replace([np.inf, -np.inf], np.nan)
55.
56.   # Drop columns with too many missing values 57.
      X = X.dropna(axis=1, thresh=int(0.7 * len(X)))
58.
59.      # Keep only numeric
60.      X = X.select_dtypes(include=np.number)
61.
62.      # Fill remaining NaN
63.      X = X.fillna(X.median())
64.      from sklearn.feature_selection import VarianceThreshold
65.
66.      selector = VarianceThreshold(threshold=0.0)
67.      X_var = selector.fit_transform(X)
68.
```

```
    cols = X.columns[selector.get_support()]
    X = pd.DataFrame(X_var, columns=cols)
    # Ensure no negative values before log
    X = X.clip(lower=0)

    # Replace inf again (safety)
    X = X.replace([np.inf, -np.inf], np.nan)

    # Fill NaN safely
```

```
78.      X = X.fillna(X.median())
79.
80.      # Apply log transform safely
81.      X_log = np.log1p(X) 82.   X_log =
         pd.DataFrame(X_log)
83.
84.      X_log = X_log.replace([np.inf, -np.inf], np.nan)
```

```
    X_log = X_log.fillna(X_log.median())
    # Convert to DataFrame (if not already)
    X_log = pd.DataFrame(X_log)

# Clip extreme values (IMPORTANT 🔥)
X_log = X_log.clip(lower=X_log.quantile(0.01),
upper=X_log.quantile(0.99),
axis=1)
from sklearn.preprocessing import QuantileTransformer
import numpy as np 95. import pandas as pd

    # Convert to DataFrame (safety)
    X_log = pd.DataFrame(X_log)

    # STEP 1: Replace bad values
    X_log = X_log.replace([np.inf, -np.inf], np.nan)

    # STEP 2: Fill missing values (IMPORTANT)
    X_log = X_log.fillna(X_log.median())

    # STEP 3: Clip extreme values (VERY IMPORTANT 🔥)
    lower = X_log.quantile(0.01)
    upper = X_log.quantile(0.99)
    X_log = X_log.clip(lower=lower, upper=upper, axis=1)

    # STEP 4: Remove near-constant columns (stability fix - increased
    threshold for robustness)
    X_log = X_log.loc[:, X_log.std() > 0.01]

    # STEP 5: Apply Quantile Transformation
    qt = QuantileTransformer(output_distribution='normal',
    random_state=42)
    X_transformed = qt.fit_transform(X_log)

    # Final dataset
    X_final = pd.DataFrame(X_transformed, columns=X_log.columns)
    print(np.isnan(X_final).sum().sum())
    print(np.isinf(X_final).sum().sum())
    skewness_final = X_final.skew().sort_values(ascending=False)
    print(skewness_final.head(10))
```

```
    skewness = X_final.skew()

    # Select features with skewness > 5
    high_skew_cols = skewness[skewness > 5].index

    print("Dropping:", list(high_skew_cols))

    # Drop them
    X_final = X_final.drop(columns=high_skew_cols)
```

```
    skewness_final = X_final.skew().sort_values(ascending=False)
```

```
    print(skewness_final.head(10))
    skewness = X_final.skew()
    high_skew_cols = skewness[skewness > 4].index

    X_final = X_final.drop(columns=high_skew_cols)
    skewness_final = X_final.skew().sort_values(ascending=False)
    print(skewness_final.head(10))
    from sklearn.preprocessing import StandardScaler

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X_final)
    from sklearn.model_selection import train_test_split

        X_train, X_test, y_train, y_test = train_test_split(
        X_scaled, y, test_size=0.2, random_state=42
        )
        from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
n_estimators=100,
max_depth=10,
random_state=42
) 157.
    model.fit(X_train, y_train)
    from sklearn.metrics import classification_report, confusion_matrix,
    accuracy_score

    y_pred = model.predict(X_test)

    print("Accuracy:", accuracy_score(y_test, y_pred))
    print(classification_report(y_test, y_pred))
    # Create a publication-quality confusion matrix figure based on
    provided values

    import numpy as np
    import matplotlib.pyplot as plt
    from sklearn.metrics import ConfusionMatrixDisplay

# Given confusion matrix values
```

```
172.  cm = np.array([[45282, 7], 173.                  [5,
14706]])
174.
175.      # Labels
176.      labels = ["Benign (0)", "Attack (1)"]
177.
178.      # Plot
179.      fig, ax = plt.subplots(figsize=(6,6))
180.      disp = ConfusionMatrixDisplay(confusion_matrix=cm,
          display_labels=labels)
181.      disp.plot(ax=ax, cmap='Blues', colorbar=True)
182.
183.      # Title and formatting
184.      plt.title("Confusion Matrix for Random Forest Model", fontsize=12)
```

```
185.     plt.grid(False)
```

```
186.     plt.tight_layout()
187.
188.     plt.show()
189.     import matplotlib.pyplot as plt
190.     import numpy as np
191.
```

```
    # Get feature importance
    importances = model.feature_importances_
    indices = np.argsort(importances)[-10:]

    features = X_final.columns[indices]
    values = importances[indices]

    plt.figure(figsize=(8,6))
    bars = plt.barh(features, values, color="steelblue",
    edgecolor="black")

# Add numeric labels at the end of each bar
for bar, val in zip(bars, values):
    plt.text(val + 0.002, bar.get_y() + bar.get_height()/2,
             f"{val:.3f}", va='center', fontsize=9)

    # Labels
    plt.xlabel("Importance")
    plt.title("Top 10 Important Features")

plt.tight_layout()
plt.show()
   print(X_train.shape)
   print(X_test.shape)
   from sklearn.svm import SVC
   from sklearn.metrics import accuracy_score, classification_report

   svm_model = SVC()

   svm_model.fit(X_train, y_train)

   y_pred_svm = svm_model.predict(X_test)

   print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))
   print(classification_report(y_test, y_pred_svm))
   !pip install xgboost
   from xgboost import XGBClassifier

xgb_model = XGBClassifier(
n_estimators=100,
```

```
232.  max_depth=6,
233.  learning_rate=0.1,
234.  random_state=42,
235.  use_label_encoder=False,
236.  eval_metric='logloss'
237.  ) 238.
239.     xgb_model.fit(X_train, y_train)
```

```
240.
241.     y_pred_xgb = xgb_model.predict(X_test)
242.
243.     print("XGBoost Accuracy:", accuracy_score(y_test, y_pred_xgb))
```

```
    print(classification_report(y_test, y_pred_xgb))
    rf_acc = accuracy_score(y_test, y_pred)
    svm_acc = accuracy_score(y_test, y_pred_svm)
    xgb_acc = accuracy_score(y_test, y_pred_xgb)

comparison = pd.DataFrame({
'Model': ['Random Forest', 'SVM', 'XGBoost'],
'Accuracy': [rf_acc, svm_acc, xgb_acc]
}) 253.
    print(comparison)
    from sklearn.metrics import precision_score, recall_score, f1_score

def get_metrics(y_true, y_pred):
return [
accuracy_score(y_true, y_pred),
precision_score(y_true, y_pred),
recall_score(y_true, y_pred),
f1_score(y_true, y_pred)
] 264.
    rf_metrics = get_metrics(y_test, y_pred)
    svm_metrics = get_metrics(y_test, y_pred_svm)
    xgb_metrics = get_metrics(y_test, y_pred_xgb)

comparison = pd.DataFrame({
'Model': ['Random Forest', 'SVM', 'XGBoost'],
'Accuracy': [rf_metrics[0], svm_metrics[0], xgb_metrics[0]],
'Precision': [rf_metrics[1], svm_metrics[1], xgb_metrics[1]],
'Recall': [rf_metrics[2], svm_metrics[2], xgb_metrics[2]],
'F1 Score': [rf_metrics[3], svm_metrics[3], xgb_metrics[3]]
}) 276.
    print(comparison)
    comparison.set_index('Model')[['Accuracy','Precision','Recall','F1
    Score']].plot(kind='bar')
    plt.title("Model Comparison")
    plt.xticks(rotation=0)
    plt.show()
    # Create a publication-quality model comparison bar chart

    import pandas as pd
    import matplotlib.pyplot as plt
    import numpy as np

        # Given data
```

```
289.         data = {
290.         "Model": ["Random Forest", "SVM", "XGBoost"],
291.         "Accuracy": [0.999800, 0.999750, 0.999867],
292.         "Precision": [0.999524, 0.999456, 0.999660],
293.         "Recall": [0.999660, 0.999524, 0.999796],
294.         "F1 Score": [0.999592, 0.999490, 0.999728]
```

```
    }

    df = pd.DataFrame(data)

    # Set index
    df.set_index("Model", inplace=True)

    # Plot
    fig, ax = plt.subplots(figsize=(8,6))
    df.plot(kind='bar', ax=ax) 305.
    # Formatting
    plt.title("Performance Comparison of Machine Learning Models",
    fontsize=12)
    plt.ylabel("Score")
    plt.xticks(rotation=0)
    plt.ylim(0.9993, 1.0)  # zoomed for clarity
    plt.legend(loc='lower right')
    plt.tight_layout() 313.
    plt.show()
    comparison.plot(x='Model', y='Accuracy', kind='bar', legend=False)

    plt.title("Accuracy Comparison of Models")
    plt.ylabel("Accuracy")
    plt.xticks(rotation=0)
    plt.show()
    # Simulate a dataset similar in structure (numeric features) to
    generate the heatmap
    import pandas as pd
    import numpy as np
    import seaborn as sns
    import matplotlib.pyplot as plt 326.
    # Create synthetic data (since original df is not available here)
    np.random.seed(42)
    data = np.random.rand(1000, 20)
    df = pd.DataFrame(data, columns=[f'Feature_{i}' for i in range(20)])

    # Compute correlation
    corr = df.corr(numeric_only=True) 334.
    # Plot heatmap as per user's code
    plt.figure(figsize=(12,10))
    sns.heatmap(corr, cmap="coolwarm", vmin=-1, vmax=1, annot=False)
    plt.title("Correlation Heatmap of Numeric Features")
    plt.tight_layout()
    plt.show()
```

```
    plt.figure(figsize=(6,4))
    df['Flow Duration'].hist(bins=50)
    plt.title("Distribution of Flow Duration")
    plt.xlabel("Value")
    plt.ylabel("Frequency")
    plt.show()
    plt.figure(figsize=(6,4))
    sns.boxplot(x=df['Flow Duration'])
    plt.title("Outlier Detection in Flow Duration")
```

```
    plt.show()
    import numpy as np   import matplotlib.pyplot as plt

```

```
354.  # Extract the data 355.
      data = df['Flow Duration']
356.
357.     plt.figure(figsize=(8,6))
358.
359.     # Use log-spaced bins from 1 up to max value
360.     bins = np.logspace(np.log10(1), np.log10(data.max()), 50)
361.
362.     plt.hist(data, bins=bins, color='steelblue', edgecolor='black')
363.
364.     # Apply log scale to x-axis
365.     plt.xscale('log')
366.
367.     # Labels and title
368.     plt.xlabel("Flow Duration (log scale)")
369.     plt.ylabel("Frequency")
370.     plt.title("Distribution of Flow Duration (Log Scale)")
371.
372.     # Add grid for clarity
373.     plt.grid(True, which="both", linestyle="--", linewidth=0.5)
374.
375.  plt.tight_layout()
376.  plt.show() 377.
378.     plt.figure(figsize=(10,8))
379.     sns.heatmap(df.corr(numeric_only=True).iloc[:20, :20],
         cmap='coolwarm')
380.     plt.title("Feature Correlation Heatmap")
381.     plt.show()
382.     # Enhancing histogram of 'Flow Duration' with mean/median lines and
         lognormal fit for publication
383.     import numpy as np
384.     import pandas as pd
385.     import matplotlib.pyplot as plt
386.     from scipy.stats import lognorm
387.     import os 388.
389.  # Ensure output directory exists 390.
      os.makedirs("/mnt/data", exist_ok=True)
391.
392.     # Simulate example data similar to 'Flow Duration' (replace with
         actual df['Flow Duration'] in real use)
393.     np.random.seed(42)
```

```
394.     simulated_data = np.random.lognormal(mean=12, sigma=1.0,
         size=100000)
395.     df = pd.DataFrame({'Flow Duration': simulated_data})
396.
397.  # Extract the data 398.
     data = df['Flow Duration']
399.
400.     # Compute mean and median
```

```
    mean_val = data.mean()
    median_val = data.median()

    # Fit a lognormal distribution
    shape, loc, scale = lognorm.fit(data, floc=0)  # Fix location to 0
```

```
  for standard lognormal
406.
407.     # Create log-spaced bins
408.     bins = np.logspace(np.log10(data.min()), np.log10(data.max()), 50)
409.
410.     # Plot histogram
411.     plt.figure(figsize=(10, 6))
412.     counts, bins, patches = plt.hist(data, bins=bins, density=True,
        color='skyblue', edgecolor='black', alpha=0.7, label='Histogram')
413.
414.     # Plot fitted lognormal PDF
415.     x = np.logspace(np.log10(data.min()), np.log10(data.max()), 1000)
416.     pdf = lognorm.pdf(x, shape, loc=loc, scale=scale)
417.     plt.plot(x, pdf, 'r-', lw=2, label='Fitted Lognormal PDF')
418.
419.     # Add mean and median lines
420.     plt.axvline(mean_val, color='green', linestyle='--', linewidth=2,
        label=f'Mean = {mean_val:.2e}')
421.     plt.axvline(median_val, color='purple', linestyle='-.', linewidth=2,
        label=f'Median = {median_val:.2e}')
422.
423.  # Log scale and labels
424.  plt.xscale('log')
425.  plt.xlabel("Flow Duration (log scale)")
426.  plt.ylabel("Density")
427.  plt.title("Distribution of Flow Duration with Lognormal Fit")
428.  plt.grid(True, which="both", linestyle="--", linewidth=0.5)
429.  plt.legend() 430.      plt.tight_layout()
431.
432.  # Save the figure
433.  output_path = "/mnt/data/flow_duration_histogram_lognormal.png" 434.
     plt.savefig(output_path)
435.
436.  print(f"Enhanced histogram with mean, median, and lognormal fit saved
   as {output_path}")
437.
438.     import matplotlib.pyplot as plt
439.     import seaborn as sns
440.     import numpy as np
441.
442.  # Extract the data 443.
     data = df['Flow Duration']
444.
445.     plt.figure(figsize=(8,6))
446.
```

```
    # Create boxplot
    sns.boxplot(x=data, color='steelblue')

    # Apply log scale to x-axis
    plt.xscale('log')

```

```
453.     # Labels and title
454.     plt.xlabel("Flow Duration (log scale)")
455.     plt.title("Box Plot of Flow Duration (Log Scale)")
456. 457.
         # Add grid for clarity
458.     plt.grid(True, which="both", linestyle="--", linewidth=0.5)
459. 460.
         plt.tight_layout()
461. 462.plt.show()
```